HISTORICAL TECHNICAL RECORD

Slay the Spire 2
Native iOS Port

Native execution - no streaming or emulation

Architecture, debugging notes and reproduction outline
July 2026 - English translation

Personal technical research. Use your own purchased Steam game files on your own device. No game content is distributed here.

Translation and current-status note

This is a translation of the original author's technical record. Its claims about being first, smooth performance, audio, free signing and memory budgets are historical reports, not new verification by this fork. Some procedures and counts are outdated. Current instructions and limitations are in docs/BEGINNER_GUIDE.md and docs/IOS_PERFORMANCE.md. This fork uses normal 1x timing, defaults Extras off, still has recurring hitches and has not verified free-account signing. Preserve backups; do not replace an app container merely to transfer saves.

1. Overview: what the original project achieved

The author compiled the PC version of Slay the Spire 2 (Godot + C#) into a native iPhone app, reported smooth device operation and desktop/phone save transfer, and added Restart Room without relying on Steam mods. The principal challenges were the supported iOS runtime's lack of runtime code generation and the app's memory limit.

Capability	Original reported status	Approach
Native iPhone execution	Working	Official Godot 4.5.1 engine and NativeAOT game assembly
Touch, UI and lifecycle patches	Working	Mono.Cecil static IL weaving replaces runtime Harmony patches
FMOD audio and Spine animation	Working	Locally compiled iOS GDExtensions
Desktop/phone save transfer	Working	Steam saves and Xcode container operations; use the current safer save-transfer guide
Stall-free performance	Claimed by the original author	Shader warmup and increased memory; recurring hitches remain in the current fork
Restart Room	Working	Pause-menu button plus native reload logic

2. Why this game could be ported

The original analysis relied on three conditions:

- **Engine support:** Slay the Spire 2 uses the open-source Godot 4.5.1 engine, which supports iOS export. The original text contrasted this with the earlier game's Unity engine.
- **Readable content pack:** its .pck uses the standard, unencrypted GDPC format and can be parsed and repacked.
- **Android precedent:** an unofficial community launcher already supplied touch/UI-scale logic. Its runtime patch mechanism had to be replaced for iOS.

3. Core architecture

3.1 Static IL weaving for a runtime without JIT

The Android approach patched the game dynamically with Harmony. The supported iOS NativeAOT route cannot generate or replace machine code at runtime, so that mechanism is unavailable.

A Mono.Cecil weaver inserts prefix/postfix calls into a private copy of sts2.dll before native compilation. The original report said that 15 of 16 Android patches were adapted, excluding one debugging-only patch.

```
# Weave patch hooks into the private game DLL
STS2Weaver sts2.dll hooks.dll manifest.json woven-sts2.dll
# Each manifest entry identifies a target method, prefix/postfix kind and hook.
```

3.2 NativeAOT: compiling the game into a native library

Godot's supported C# iOS pipeline uses NativeAOT, compiling the game and required .NET runtime code into a dylib. With no game source available, an empty driver project creates a placeholder assembly, then replaces it with the woven DLL before ILC reads it.

Original reflection test

The game registers 1,624 card/creature types through reflection. Reflection and trimming are a major risk for AOT ports. The original author reported successful compilation and runtime operation; this does not eliminate the need to investigate warnings and test all relevant paths.

3.3 Native audio and animation extensions

The game uses FMOD and Spine GDExtensions; the owned desktop package contains desktop libraries. The original build compiled iOS arm64 counterparts matching FMOD 2.03.14 and Spine 4.2, then checked initializer symbols. The current pinned recipe separately records the versions actually tested by this fork.

3.4 Two content-pack adjustments

- **Disable Sentry:** the original package has no matching iOS extension in this route, so remove its extension registration and disable its autoload in the private pack.
- **Add C# resource placeholders:** the original report needed 672 .cs paths absent from the pack. The official engine then resolves scene script resources to precompiled types. Counts vary with game versions.

4. Debugging record

The original author used device logs and crash reports to investigate each startup failure.

Symptom	Cause identified in the original test	Resolution
Duplicate IntPtr definition at compile time	Copied the game's bundled .NET runtime assemblies alongside the compiler's runtime pack	Exclude framework libraries using the installed runtime's actual file list
ILC rejects a Steamworks type	Explicit-layout union overlapping references and numbers	Avoid rooting every dependency; allow unreachable desktop-only types to be trimmed
Logo hang / hostfxr loading failure	Main native library not embedded as an app framework	Package/embed the AOT output as the expected framework
Fatal atlas-loading error	Reflection-based JSON disabled by AOT defaults	Enable JsonSerializerIsReflectionEnabledByDefault
Scene script-loading crash	Missing .cs resource paths	Add placeholders; the original count was 672
FMOD calls a null initializer	Missing load_all_fmod_plugins symbol in the static route	Add an empty custom-plugin loader
ModelDb initialization hang	Reflection workaround for a desktop issue hung under AOT; that issue was absent on iOS	Remove the workaround and retain the game's original logic

Three symptoms attributed to one memory problem

The original report grouped Save and Quit crashes, shader-warmup crashes and some stutters under per-process memory termination, citing JetsamEvent with largestProcess: StS2 and reason: per-process-limit. It reported improvement with increased-memory-limit and estimated a limit near 75% of device RAM. Memory availability varies by device/state; this is not a guaranteed allocation or proof that every stall is fixed. The original free-account claim has not been reproduced by this fork; the actual signed profile must support the entitlement.

5. Restart Room

The feature resembled the Steam mod Quick Restart 2, but was implemented directly because its mod-loader/runtime-patching/workshop dependencies did not transfer to this iOS route.

- **Button:** statically hook pause-menu initialization, duplicate Save and Quit to retain native styling/layout, relabel it Restart Room, and place it below Continue.
- **Reload:** reuse the return-to-menu/continue path without saving. The existing room-entry autosave remains unchanged, effectively undoing the room's later actions. A black transition hides the intermediate menu.

6. Shader warmup

The original approach rendered materials in a hidden viewport on first launch to compile/cache shaders, then wrote a completion marker to skip later warmups. It aimed to reduce first-use compilation stalls. Warmup consumes additional memory; it does not prove every future effect is cached or prevent all recurring frame hitches.

7. Reproduction and maintenance

These are explanatory commands, not a complete setup script

The process also requires STS2Weaver, the mobile patches, pack tools and export configuration supplied by the repository. The commands illustrate stages and parameters; copying them without the source/configuration is insufficient. Use the current beginner guide and build script for installation.

7.1 Original version requirements

Component	Original version	Notes
Godot mono	4.5.1-stable	Match the required engine/export templates
.NET SDK	9.0.x	Game TFM net9.0 and compatible ILCompiler
FMOD Engine	2.03.14 iOS	Compatible bank format; obtain from FMOD yourself
Spine runtimes	4.2 branch	Match the game's .spskel data version
Xcode / iOS SDK	Xcode 26.x	NativeAOT cross-compiler/linker and SDK

7.2 Prerequisites

- Your purchased macOS Steam game files.
- A Mac, Xcode, iPhone and your own Apple Account. Free-account signing remains unverified by this fork.
- The original tool list used brew install scons xcodegen ios-deploy libimobiledevice, plus a .NET SDK from Microsoft. Use the current guide's actual prerequisites instead of assuming every historical tool is required.

7.3 Build stages

```
# 1. Compile patches and statically weave into a private game DLL.
dotnet build src/STS2MobileIos -c Release -o /tmp/patch
dotnet run --project src/STS2Weaver -c Release -- \
  game-refs/sts2.dll /tmp/patch/STS2MobileIos.dll \
  src/STS2MobileIos/manifest.json /tmp/sts2-woven.dll
# 2. Assemble third-party managed dependencies in prebuilt/;
#    exclude framework libraries using the installed runtime list.
# 3. Compile the iOS arm64 FMOD and Spine extensions.
scons platform=ios arch=arm64 target=template_release fmod_lib_dir=...
./build-extension.sh ios arm64 # Original Spine example; setup first.
# 4. NativeAOT: woven game -> native dylib (~75MB in the original build).
dotnet publish ios-export/sts2.csproj -c ExportRelease -r ios-arm64 \
  -p:GodotTargetPlatform=ios --self-contained
# 5. Export Xcode and prepare the pack: disable Sentry, add placeholders.
# 6. Add the empty load_all_fmod_plugins loader to generated dummy.cpp.
```

7.4 Signing and installing

The original author provisioned Increased Memory Limit through Xcode's Signing & Capabilities and reported command-line account-visibility problems. This is an observation about that setup, not a universal ban on command-line signing. Select your own Team, confirm the capability/profile and build for your iPhone. Subsequent incremental signing still needs the correct profile and identity.

```
# Initial Xcode setup:
# Open build/StS2.xcodeproj -> Signing & Capabilities
# Select your Team -> Increased Memory Limit -> Run
# Later incremental example (sign frameworks too):
cp publish/sts2.dylib "$APP/Frameworks/sts2.framework/sts2"
codesign -f -s "Apple Development: YOUR_ACCOUNT" \
  --generate-entitlement-der --entitlements ent_mem.plist "$APP"
xcrun devicectl device install app --device <UDID> "$APP"
# Install over the existing same-bundle app; never uninstall to update.
```

Three maintenance rules

- Install over the same app. Uninstalling deletes its local saves.
- Free Personal Team signing normally expires after 7 days; renew through Xcode using the same identity/bundle. Other profiles can differ.
- Rebuild after game updates. The original note expected shader-cache reset and another first-start warmup; verify actual cache behavior for your build.

Save transfer

The original report located Steam saves under userdata/<id>/2868840/remote/ and described Xcode container operations into Documents/default/1/. Do not blindly replace the current app container: use docs/SAVE_SYNC.md, independently back up both sides, and use the validated transfer/recovery scripts.

Personal technical research, unaffiliated with Mega Crit / MegaCrit. Slay the Spire 2 and its content belong to their respective rights holders.